Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Шумихина М.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.10.26

Москва, 2026

Постановка задачи

Вариант 3.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить. Отсортировать массив целых чисел при помощи параллельной сортировки слиянием.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pthread_create - создаёт новый поток выполнения;
- pthread_join - блокирует родительский поток до завершения дочернего;
- pthread_mutex_lock / pthread_mutex_unlock - захватывают и освобождают мьютекс
- malloc - выделяет память под массивы данных и под структуры аргументов ThreadArgs;
- free - освобождает память, выделенную под массивы и структуры аргументов, после завершения работы потока;
- printf - форматированный вывод;
- rand, srand - генерация исходного массива случайных чисел;
- memcpy - копирование исходного массива;
- clock_gettime - возвращает текущее время;
- usleep, sleep - используются для задержки потоков.

main читает ключ -p N (ограничение максимального количества потоков, задаётся ключом запуска) и необязательный размер массива. Без ключа -p программа строит таблицу ускорения для набора p={1, 2, 3, 4, 16, 128, 1024}, с ключом программа работает только с заданным p для демонстрации потоков средствами ОС. Число потоков ограничивается переменной max_threads_limit.

Функция sequential_mergesort(arr, low, high, temp) - классическая сортировка слиянием: массив делится пополам (mid = low + (high - low) / 2), рекурсивно сортируются левая и правая части, затем выполняется слияние через вспомогательный массив temp. Время мерится через clock_gettime до и после; делается 3 замера, берётся минимум.

Функция parallel_mergesort_thread(void *_args) получает структуру ThreadArgs с полями arr, low, high, temp. После разбиения массива поток проверяет под мьютексом, можно ли создать ещё два потока. Если да, создаёт два дочерних потока для левой и правой частей через pthread_create и ждёт их через pthread_join. Если лимит исчерпан, сортирует оставшиеся части последовательно через sequential_mergesort. Затем выполняется слияние.

Подсчёт метрик. S(p) = T_seq / T_par(p) - во сколько раз параллельная версия быстрее последовательной. E(p) = S(p) / p - доля полезной загрузки каждого потока (E = 1 - идеал, E → 0 - потоки простаивают).

Код программы

```c
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <pthread.h>


static pthread_mutex_t thread_count_mutex = PTHREAD_MUTEX_INITIALIZER;
static size_t active_threads = 0;
static size_t max_threads_limit = 1;
static bool verbose_threads = false;
static bool demo_mode = false;


double get_time_ms() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1000000.0;
}


void merge(int32_t *arr, int32_t low, int32_t mid, int32_t high, int32_t *temp) {
    int32_t i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= high) temp[k++] = arr[j++];
    for (i = low; i <= high; i++) arr[i] = temp[i];
}
```

```c
void sequential_mergesort(int32_t *arr, int32_t low, int32_t high, int32_t *temp)
{
    if (low < high) {
        int32_t mid = low + (high - low) / 2;
        sequential_mergesort(arr, low, mid, temp);
        sequential_mergesort(arr, mid + 1, high, temp);
        merge(arr, low, mid, high, temp);
    }
}

static void print_array(const char *label, int32_t *arr, int32_t n) {
    printf("%s: [", label);
    for (int32_t i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) printf(", ");
    }
    printf("]\n");
}

typedef struct {
    int32_t *arr;
    int32_t low;
    int32_t high;
    int32_t *temp;
} ThreadArgs;

static void *parallel_mergesort_thread(void *_args) {
    ThreadArgs *args = (ThreadArgs *)_args;
    int32_t *arr = args->arr;
    int32_t low = args->low;
    int32_t high = args->high;
    int32_t *temp = args->temp;
```

```
        free(args);


        if (verbose_threads) {
            printf("  -> поток %lu начал работу с диапазоном [%d..%d]\n",
                   (unsigned long)pthread_self(), low, high);
        }


        if (demo_mode) {
            usleep(4000000);
        }


        pthread_mutex_lock(&thread_count_mutex);
        active_threads++;
        pthread_mutex_unlock(&thread_count_mutex);


        if (low < high) {
            int32_t mid = low + (high - low) / 2;
            bool can_create = false;


            pthread_mutex_lock(&thread_count_mutex);
            if (active_threads + 1 < max_threads_limit) {
                active_threads += 2;
                can_create = true;
            }
            pthread_mutex_unlock(&thread_count_mutex);


            if (can_create) {
                pthread_t left_thread, right_thread;
                ThreadArgs *left_args = malloc(sizeof(ThreadArgs));
                *left_args = (ThreadArgs){ .arr = arr, .low = low, .high = mid, .temp
= temp };
                ThreadArgs *right_args = malloc(sizeof(ThreadArgs));
```

```
                *right_args = (ThreadArgs){ .arr = arr, .low = mid + 1, .high =
high, .temp = temp };



                pthread_create(&left_thread, NULL, parallel_mergesort_thread,
left_args);
                pthread_create(&right_thread, NULL, parallel_mergesort_thread,
right_args);
                pthread_join(left_thread, NULL);
                pthread_join(right_thread, NULL);
            } else {
                sequential_mergesort(arr, low, mid, temp);
                sequential_mergesort(arr, mid + 1, high, temp);
            }
            merge(arr, low, mid, high, temp);
        }



        pthread_mutex_lock(&thread_count_mutex);
        active_threads--;
        pthread_mutex_unlock(&thread_count_mutex);



        return NULL;
    }



    static void demo_small_array(size_t p) {
        const int32_t DEMO_N = 8;
        int32_t demo[8];
        int32_t temp[8];



        srand(123);
        for (int32_t i = 0; i < DEMO_N; i++) demo[i] = rand() % 100;



        printf("\nДемонстрация сортировки на массиве из %d элементов:\n", DEMO_N);
        print_array("До сортировки   ", demo, DEMO_N);
```

```
        max_threads_limit = p;
        active_threads = 0;
        verbose_threads = true;
        demo_mode = false;

        ThreadArgs *args = malloc(sizeof(ThreadArgs));
        *args = (ThreadArgs){ .arr = demo, .low = 0, .high = DEMO_N - 1, .temp =
temp };

        pthread_t main_thread;
        pthread_create(&main_thread, NULL, parallel_mergesort_thread, args);
        pthread_join(main_thread, NULL);

        verbose_threads = false;
        print_array("После сортировки", demo, DEMO_N);
    }

    int main(int argc, char **argv) {
        int32_t N = 1000000;
        size_t single_p = 0;

        for (int i = 1; i < argc; i++) {
            if (strcmp(argv[i], "-p") == 0 && i + 1 < argc) {
                single_p = (size_t)atoi(argv[++i]);
            } else {
                int32_t val = atoi(argv[i]);
                if (val > 0) N = val;
            }
        }
```

```
        int32_t *template_arr = malloc(N * sizeof(int32_t));
        int32_t *arr_seq = malloc(N * sizeof(int32_t));
        int32_t *arr_par = malloc(N * sizeof(int32_t));
        int32_t *temp = malloc(N * sizeof(int32_t));

        srand(42);
        for (int32_t i = 0; i < N; i++) template_arr[i] = rand() % 10000;

        double seq_time = 1e30;
        for (int rep = 0; rep < 3; rep++) {
            memcpy(arr_seq, template_arr, N * sizeof(int32_t));
            double t0 = get_time_ms();
            sequential_mergesort(arr_seq, 0, N - 1, temp);
            double t1 = get_time_ms();
            double dt = t1 - t0;
            if (dt < seq_time) seq_time = dt;
        }

        if (single_p > 0) {
            demo_mode = true;
            max_threads_limit = single_p + 1;

            memcpy(arr_par, template_arr, N * sizeof(int32_t));
            ThreadArgs *args = malloc(sizeof(ThreadArgs));
            *args = (ThreadArgs){ .arr = arr_par, .low = 0, .high = N - 1, .temp =
temp };

            pthread_mutex_lock(&thread_count_mutex);
            active_threads = 0;
            pthread_mutex_unlock(&thread_count_mutex);

            pthread_t main_thread;
```

```
            pthread_create(&main_thread, NULL, parallel_mergesort_thread, args);


            printf("Все %zu потоков запущены. 5 секунд на просмотр потоков:\n",
single_p);
            printf("  ps -eLf | grep lab2\n\n");
            sleep(5);


            pthread_join(main_thread, NULL);


            printf("Сортировка завершена.\n");


            free(args);
        } else {
            size_t test_threads[] = {1, 2, 3, 4, 16, 128, 1024};
            size_t num_tests = sizeof(test_threads) / sizeof(test_threads[0]);


            printf("| Threads      | Time (ms)   | Speedup | Efficiency |\n");
            printf("+--------------+-------------+---------+------------+\n");
            printf("| 1 (seq)      | %11.2f | %7.2f | %10.3f |\n", seq_time, 1.0,
1.0);


            size_t best_p = 1;
            double best_time = seq_time;


            for (size_t t = 0; t < num_tests; t++) {
                size_t p = test_threads[t];
                max_threads_limit = p;


                memcpy(arr_par, template_arr, N * sizeof(int32_t));
                ThreadArgs *warm_args = malloc(sizeof(ThreadArgs));
```

```
                *warm_args = (ThreadArgs){ .arr = arr_par, .low = 0, .high = N -
1, .temp = temp };
                pthread_t warm_thread;
                pthread_create(&warm_thread, NULL, parallel_mergesort_thread,
warm_args);
                pthread_join(warm_thread, NULL);



                double par_time = 1e30;
                for (int rep = 0; rep < 3; rep++) {
                    memcpy(arr_par, template_arr, N * sizeof(int32_t));
                    ThreadArgs *args = malloc(sizeof(ThreadArgs));
                    *args = (ThreadArgs){ .arr = arr_par, .low = 0, .high = N -
1, .temp = temp };



                    pthread_mutex_lock(&thread_count_mutex);
                    active_threads = 0;
                    pthread_mutex_unlock(&thread_count_mutex);



                    pthread_t main_thread;
                    double t0 = get_time_ms();
                    pthread_create(&main_thread, NULL, parallel_mergesort_thread,
args);
                    pthread_join(main_thread, NULL);
                    double t1 = get_time_ms();



                    double dt = t1 - t0;
                    if (dt < par_time) par_time = dt;
                }



                double speedup = seq_time / par_time;
                double efficiency = speedup / p;



                printf("| %-12zu | %11.2f | %7.2f | %10.3f |\n",
```

```
                    p, par_time, speedup, efficiency);


            if (par_time < best_time) {
                best_time = par_time;
                best_p = p;
            }
        }
        printf("+--------------+-------------+---------+------------+\n");
        demo_small_array(best_p > 0 ? best_p : 4);
    }


    free(template_arr);
    free(arr_seq);
    free(arr_par);
    free(temp);
    return 0;
}
```

Протокол работы программы

```
| Threads      | Time (ms)   | Speedup | Efficiency |
+--------------+-------------+---------+------------+
| 1 (seq)      |      340.12 |    1.00 |      1.000 |
| 1            |      375.00 |    0.91 |      0.907 |
| 2            |      312.28 |    1.09 |      0.545 |
| 3            |      170.04 |    2.00 |      0.667 |
| 4            |      174.51 |    1.95 |      0.487 |
| 16           |      100.31 |    3.39 |      0.212 |
| 128          |      127.68 |    2.66 |      0.021 |
| 1024         |      204.43 |    1.66 |      0.002 |
+--------------+-------------+---------+------------+

Демонстрация сортировки на массиве из 8 элементов:
До сортировки   : [93, 13, 73, 30, 79, 31, 95, 22]
  -> поток 134960512743104 начал работу с диапазоном [0..7]
  -> поток 134958165173952 начал работу с диапазоном [0..3]
  -> поток 134960504350400 начал работу с диапазоном [2..3]
  -> поток 134960421795520 начал работу с диапазоном [2..2]
  -> поток 134960413402816 начал работу с диапазоном [3..3]
  -> поток 134960405010112 начал работу с диапазоном [0..1]
  -> поток 134960421795520 начал работу с диапазоном [1..1]
  -> поток 134960379832000 начал работу с диапазоном [4..7]
  -> поток 134960388224704 начал работу с диапазоном [6..7]
  -> поток 134960413402816 начал работу с диапазоном [0..0]
  -> поток 134960396617408 начал работу с диапазоном [4..5]
  -> поток 134960421795520 начал работу с диапазоном [4..4]
  -> поток 134960413402816 начал работу с диапазоном [5..5]
После сортировки: [13, 22, 30, 31, 73, 79, 93, 95]
```

Вывод

Анализ результатов на системе с 4 логическими ядрами показал следующее:

1. При p=1 и p=2 (меньше числа ядер) ускорение невелико, что объясняется накладными расходами на создание потоков и ожидание.

2. При p=3 и p=4 (меньше или равно числу ядер) достигается наилучшая эффективность (0.667 и 0.487 соответственно). Ускорение при p=4 составляет около 1.95, что близко к теоретическому пределу для сортировки слиянием на 4 ядрах.

3. При p=16, 128, и 1024 (значительно больше числа ядер) наблюдается резкое падение эффективности (до 0.021 и 0.002). Здесь количество потоков превышает число физических ядер, и они начинают конкурировать за процессорное время. Операционная система тратит ресурсы на переключение контекста, а не на полезную сортировку.

4. Небольшие колебания ускорения в таблице (например, скачок при p=16) объясняются фоновым шумом ОС и коротким временем работы алгоритма (сотни миллисекунд).

Лично для меня эта лабораторная работа показалась очень сложной. Долго не могла разобраться в построении таблицы для всех экспериментов, в том, как именно рассчитывать ускорение и эффективность, и как правильно управлять несколькими потоками. Однако, несмотря на все трудности, я успешно овладела навыками многопоточного программирования, научилась синхронизировать потоки с помощью мьютексов и применять модель Fork-Join.